

Теми за Държавен Изпит

Д. Иванов, КН

11 юли 2025 г.

Процедурно програмиране

14. Процедурно програмиране – основни конструкции.

1. Основни принципи

Основните принципи на процедурното програмиране са принципа за **модулност** и принципа за **абстракция на данните**. Съгласно принципа за модулност, програмата се разделя на подходящи взаимосвързани части (модули), които могат да бъдат проектирани, реализирани и тествани независимо една от друга. Съгласно принципа за абстракция на данните, методите за използване на данните се отделят от методите за тяхното коректно представяне. Програмите се конструират така, че да работят с абстрактни данни - данни с неуточнено представяне.

2. Управление на изчислителния процес

Управлението на изчислителния процес се осъществява чрез управляващи конструкции, които определят реда и условията за изпълнение на операторите в програмата. Те включват условни и циклични оператори, които позволяват разклоняване и повторение на части от програмния код според дадени условия.

Условни оператори - чрез тези оператори се реализират разклоняващи се изчислителни процеси. Оператор, който дава възможност да се изпълни (или не) един или друг оператор в зависимост от някакво условие, се нарича условен. Примери за условни оператори са: **if**, **if/else** и **switch**.

- **if** оператор

Синтаксис

```
1 // if (<условие>) <оператор>
```

където **if** е запазена дума, <условие> е булев израз и <оператор> е произволен оператор.

Семантика

Пресмята се стойността на булевия израз, представящ условието. Ако резултатът е **true**, се изпълнява <оператор>, в противен случай не се изпълнява.

- **if/else** оператор

Синтаксис

```
1 // if (<условие>) <оператор1> else <оператор2>
```

където **if** и **else** са запазени думи, <условие> е булев израз и <оператор1> и <оператор2> са произволни оператори.

Семантика

Пресмята се стойността на булевия израз, представящ условието. Ако резултатът е **true**, се изпълнява <оператор1>, в противен случай се изпълнява <оператор2>.

- **switch** оператор

Синтаксис

```
1 // switch (<израз>)
2 // case <израз_1> : <редица_от_оператори_1>
3 // ...
4 // case <израз_n-1> : <редица_от_оператори_n-1>
5 // [default : <редица_от_оператори_n>]
```

където **switch**, **case** и **default** са запазени думи, <израз> е израз от допустим тип (**int**, **bool**, **char**), <израз_1>, ..., <израз_n> са константни изрази, задължително с различни стойности, а <редица_от_оператори> се дефинира така:

<редица_от_оператори> ::= <празно> | оператор
| <оператор><редица_от_оператори>

Семантика

Намира се стойността на **switch** израза, получената константа се сравнява последователно със стойностите на етикетите израз₁...израз_n и при съвпадение, се изпълняват операторите на съответния вариант и операторите на всички варианти, разположени след него, до срещане на оператор **break**. В противен случай, ако участва **default**-вариант, се изпълнява редицата от оператори, която му съответства и в случай, че не участва такъв - не следват никакви действия от оператора **switch**.

Оператори за цикъл - използват се за реализиране на циклични изчислителни процеси. Изчислителен процес, при който оператор или група от оператори се изпълняват многократно за различни стойности на техни параметри, се нарича цикличен. Съществуват два вида изчислителни процеси - индуктивни, при които броят на повторенията е известен предварително и итеративни, при които броят на повторенията не е известен предварително.

- **for** оператор

Синтаксис

```
1 // for (<инициализация>; <условие>; <корекция>) <оператор>
```

където **for** е запазена дума, <условие> е булев израз, <инициализация> е или точно една дефиниция с инициализация на една или повече променливи, или няколко оператора за присвояване или въвеждане, отделени със , и не завърващи с ;, <корекция> е един или няколко оператора, незавърващи с ; (в случай че са няколко се отделят със ,) и <оператор> е точно един произволен оператор, който се нарича тяло на цикъла.

Семантика

Изпълнението започва с изпълнение на частта <инициализация>. След това се намира стойността на <условие>. Ако в резултат се е получило **false**, изпълнението на оператора **for** завършва, без тялото да се е изпълнило нито веднъж. В противен случай последователно се повтарят следните действия: изпълнение на тялото на цикъла, изпълнение на операторите от частта <корекция>, пресмятане на стойността на <условие> докато стойността на <условие> е **true**.

- **while** оператор

Синтаксис

```
1 // while (<условие>) <оператор>
```

където **while** е запазена дума, <условие> е булев израз и <оператор> е произволен оператор, който описва действията, които се повтарят и се нарича тяло на цикъла.

Семантика

Пресмята се стойността на <условие>. Ако тя е **false**, изпълнението на оператора **while** завършва без да се е изпълнило тялото му нито веднъж. В противен случай, изпълнението на <оператор> и пресмятане на стойността на <условие> се повтарят докато <условие> е **true**. В първия момент, когато <условие> стане **false**, изпълнението на **while** завършва.

- **do/while** оператор

Синтаксис

```
1 // do <оператор> while (<условие>)
```

където **do** и **while** са запазени думи, <условие> е булев израз и <оператор> е произволен оператор, който описва действията, които се повтарят и се нарича тяло на цикъла.

Семантика

Изпълнява се тялото на цикъла, след което се пресмята стойността на <условие>. Ако то е **false**, изпълнението на оператора **do/while**

завършва. В противен случай, се повтарят действията: изпълнение на тялото и пресмятане на стойността на <условие>, докато стойността на <условие> е **true**. Веднага, след като стойността му стане **false**, изпълнението на оператора завършва.

3.Променливи

Променливата е място за съхранение на данни, което може да съдържа различни стойности по време на изпълнение на програмата. Означава се чрез редица от букви, цифри и долна черта, започваща с буква или долна черта. Променливите имат три характеристики: тип, име и стойност. Преди да бъдат използвани, те трябва да бъдат дефинирани. C++ е строго типизиран език за програмиране. Всяка променлива има тип, който явно се указва при дефинирането ѝ.

Дефиниране на променливи

Синтаксис:

```
1 // <име_на_тип> <променлива> [ = <израз> ] {, <променлива>  
2 // [ = <израз> ] };
```

където <име_на_тип> е дума, означаваща име на тип като **int**, **double** и др., а <израз> е правило за получаване на стойност - цяла, реална, знакова и др. тип, съвместим с <име_на_тип>.

Семантика:

Дефиницията свързва променливата с множеството от допустимите стойности на типа, от който е променливата или с конкретна стойност от това множество. За целта се отделя определено количество оперативна памет (толкова, колкото да се запише най-голямата константа от множеството от допустимите стойности на съответния тип) и се именува с името на променливата. Тази памет е с неопределено съдържание или съдържа стойността на указания израз, ако е направена инициализация. Не се допуска една и съща променлива да има няколко дефиниции в рамките на една и съща функция.

Видове променливи

- Локални - променливи, които имат локална област на действие. Те са дефинирани във функции и не са достъпни за кода в другите функции на модула. Областта им се определя според общо правило – започва от мястото на дефинирането и завършва в края на оператора (блока), в който идентификаторът е дефиниран. Формалните параметри на функциите също имат локална видимост. Областта им е тялото на функцията.
- Глобални - променливи, които се дефинират извън функциите и които са "видими" за всички функции, дефинирани след тях. Декларират се

както се дефинират другите (локалните) променливи. Използването на много глобални променливи е лош стил за програмиране и не се препоръчва. Всяка глобална променлива трябва да е съпроводена с коментар, обясняващ предназначението ѝ.

Инициализация на променлива

Това е процес на присвояване на начална стойност на дадена променлива в момента на нейното създаване. Пример:

```
1 int x = 5; // декларация и инициализация на x
```

Инициализирането на променливи се използва, за да се избегнат непредвидими стойности, които могат да бъдат автоматично присвоени от програмата.

Оператор за присвояване

Синтаксис:

```
1 // <променлива> = <израз>;
```

където <променлива> и <израз> са от един и същ тип.

Семантика:

Пресмята стойността на <израз> и я записва в паметта, именувана с променливата от лявата страна на знака за присвояване =.

Пример:

```
1 a = 1;  
2 b = 2;  
3 c = 2 * (a + b);
```

Чрез операторите за присвояване, променливите a , b и c получават текущи стойности.

4. Функции, процедури и параметри

Функциите са основни структурни единици, които изграждат програмите на езика. Сред функциите задължително трябва да има точно една с име `main`, наречена главна функция. Тя е първата функция, която се изпълнява при стартиране на програмата. Използването на функции има следните предимства:

- програмите стават ясни и лесни за тестване и модифициране
- избягва се многократно повторение на едни и същи програмни фрагменти, които се дефинират еднократно и могат да се изпълняват произволен брой пъти

- постига се икономия на памет, тъй като кодът на функцията се съхранява само на едно място в паметта, независимо от броя на нейните изпълнения

Разпределението на оперативната памет за една програма най-общо се състои от:

- програмен код - записан е изпълнимият код на всички функции, изграждащи потребителската програма
- област на статичните данни - записани са глобалните обекти на програмата
- област на динамичните данни - за реализиране на динамични структури от данни чрез използване на средства за динамично разпределение на паметта. Чрез тях се заделя и освобождава памет в процеса на изпълнение на програмата, която е областта на динамичните данни
- програмен стек - вид памет, която съхранява данните на функциите на програмата.

Стекът е динамична структура, организирана по правилото *LIFO*, представляваща редица от елементи с пряк достъп до елементите от единия си край, наречен връх. Достъпът се реализира чрез указател. Операцията включване се осъществява само пред елемента от върха, а операцията изключване - само за елемента от върха. Елементите на програмния стек са "блокове" от памет, съхраняващи данни, дефинирани в някоя функция, които се наричат стекови рамки.

Пример за функция:

```

1 int gcd(int a, int b) {
2     while (a != b) {
3         (a > b) ? a = a - b : b = b - a;
4     }
5
6     return a;
7 }
8
9 int main() {
10     int x = 5, y = 10;
11
12     int result = gcd(x, y);
13
14     cout << "GCD of" << x << " and " << y << " is: "
15         << result;
16
17     return 0;
18 }

```

Двуаргументната функция с име `gcd`, е от тип `int`, а параметрите `a` и `b` се наричат **формални параметри** на функцията. Тялото и е блок, реализиращ алгоритъма на Евклид за намиране на НОД на две числа, който завършва с оператора `return`. Чрез него се прекратява изпълнението на функцията като стойността на израза след `return` се връща като стойност на `gcd`.

Дефинициите на функциите `main` и `gcd` се записват в областта на паметта, определена за програмния код. Изпълнението на програмата започва с изпълнение на функцията `main`. В нея се дефинират и инициализират стойностите `x` и `y`. Те се записват в дъното на програмния стек. Така на дъното на стека се оформя "блок" от памет за `main`, който се нарича **стекова рамка** на функцията `main`. Операторът `int result = gcd(x, y)` дефинира променлива `result` като в стековата рамка на `main`, веднага след променливата `y` отделя $4B$, в които ще се запише резултатът от обръщението към функцията `gcd`. Променливите `x` и `y` се наричат **фактически параметри** на това обръщение. Обръщението към `gcd` работи с копия на стойностите `x` и `y`, запомнени в `a` и `b`, а не със самите `x` и `y`. В процеса на изпълнение на тялото на `gcd`, стойностите на `a` и `b` се променят, но това не оказва влияние върху стойностите на фактическите параметри `x` и `y`. Такова свързване на формалните с фактическите параметри се нарича свързване по стойност или предаване на параметри по стойност.

Нека сега да разгледаме същия код, но със следната синтактична промяна:

```
1 int gcd(int& a, int& b) {  
2     ...  
3 }  
4  
5 int main() {  
6     ...  
7 }
```

Тук свързването на формалните с фактическите параметри се нарича свързване по име или предаване на параметри по референция. Това означава, че вътре в тялото на функцията се работи със същите променливи, които са подадени от извикващия код, т.е. `a` и `b` вече не са копия, а директни препратки към оригиналните `x` и `y` и затова всяка промяна на `a` и `b` в `gcd` се отразява на стойностите на `x` и `y` в `main`.

При извикване на функция в C++ се извършва проверка дали типовете на подадените аргументи (фактически параметри) съответстват на типовете на формалните параметри, декларирани във функцията. Ако има точно съвпадение, функцията се извиква директно. Ако типът е съвместим, но не идентичен (напр. `int` $\rightarrow$ `double`), се прави неявно преобразуване. При несъвместим тип, компилаторът извежда грешка.

Пример:

```

1 void printNumber(int n) {
2     cout << n;
3 }
4
5 printNumber(5);           // OK - int
6 printNumber(3.14);        // OK - double → int (ще се закръгли)
7 printNumber("hello");     // ГРЕШКА - string не може да се
8 //преобразува в int

```

5. Символни низове

Логическо описание

Редица от краен брой символи, заградени в кавички, се нарича символен низ или само низ. Броят на символите в редицата се нарича дължина на низ. Низ, който се съдържа в друг низ, се нарича негов подниз. Конкатенация на два низа е низ, получен като в края на първия низ се запише вторият. Два символни низа се сравняват лексикографски - сравнява се всеки символ от първия низ със символа на другия низ на същата позиция, докато не се намерят два различни символа или до края на поне един от символните низове. Ако кодът на символ от първия низ е по-малък от кода на съответния символ от втория низ се приема, че първият низ е по-малък от втория.

Физическо представяне

В оперативната памет, низовете се представят последователно. Съществуват 2 начина за разглеждане на низовете в C++:

- масиви от символи
- указатели към тип `char`

Ще разгледаме символните низове като масиви от символи.

```

1 char str[100]; // дефинира променливата str като масив от 100
2 // символа
3 char str2[5] = {'a', 'b', 'c'} // дефинира масив от символи
4 // str2 и го инициализира, тук тъй като при инициализацията са
5 // указани по-малко от 5 символа, останалите се допълват с '0'

```

Низовете разглеждаме като редица от символи, завършващи с нулевия символ `\0`, наречен още терминираща нула. Предимството на това е, че не е необходимо за всеки низ да се пази в променлива дължината му, тъй като знакът за края на низ позволява да се определи краят му. Дефиницията:

```

1 char s[5] = {'a', 'b', 'b', 'a', '\0'};

```

е еквивалентна на:

```

1 char s[5] = "abba";

```


Тоест езикът ни позволява друг начин за запис на масиви от символи, при който знакът за край на низ `\0` не се включва явно.

Основни операции и вградени функции:

- извеждане на низ

```
1 cout << str; // извежда низа, който е стойност на s.  
2 // Не е нужно да се грижим за дължината му, защото  
3 // '0' идентифицира края му.
```

- дължина на низ (`strlen`)

```
1 strlen(str); // където str е произволен низ
```

Намира дължината на `<str>`.

- конкатенация на низове (`strcat`)

```
1 strcat(dest_str, src_str) // където dest_str е  
2 // променлива от тип низ, а src_str е низ
```

Конкатенира низа, който е стойност на `dest_str` с низа `src_str`. Резултатът от конкатенацията се връща от функцията, а също се съдържа в променливата `dest_str`.

- копиране на низ (`strcpy`)

```
1 strcpy(dest_str, src_str) // където dest_str е променлива  
2 // от тип низ, а src_str е низ
```

Копира `src_str` в `dest_str`. Ако `src_str` е по-дълъг от допустимата за `dest_str` дължина, са възможни труднооткриваеми грешки. Резултатът от копирането се връща от функцията, а също се съдържа в `dest_str`.

- сравняване на низове (`strcmp`)

```
1 strcmp(str1, str2) // където str1 и str2 са низове.
```

Низовете `str1` и `str2` се сравняват лексикографски. Функцията `strcmp` е целочислена. Резултатът от обръщение към нея е цяло число с отрицателна стойност, ако `str1` е по-малък от `str2`, 0 ако `str1` е равен на `str2` и положителна стойност, ако `str1` е по-голям от `str2`.

- търсене на низ в друг (`strstr`)

```
1 strstr(str1, str2) // където str1 и str2 са  
2 // произволни низове
```

Търси `str2` в `str1`. Ако `str2` се съдържа в `str1`, `strstr` връща подниза на `str1` започващ от първото срещане на `str2` до края на `str1`. Ако `str2` не се съдържа в `str1`, `strstr` връща `nullptr`. Последното означава, че в позиция на условие, функционалното обръщение ще има стойност `false`, но при опит за извеждане, ще предизвика грешка.

- търсене на символ в низ (`strchr`)

```
1 strchr(str, expr) // където str е произволен низ, а
2 // expr е израз от интегрален или изброен тип с положителна
3 // стойност, означаваща ASCII код на символ.
```

Търси първото срещане на символа, чийто ASCII код е равен на стойността на `expr`. Ако символът се среща, функцията връща подниза на `str` започващ от първото срещане на символа и продължаващ до края му. Ако символът не се среща - връща `nullptr`. Последното означава, че в позиция на условие, функционалното обръщение ще има стойност `false`, но при опит за извеждане, ще предизвика грешка.

15. Процедурно програмиране – указатели, масиви и рекурсия.

1. Указатели и указателна аритметика

Тип указател

Променливите са свързани със стойности - неопределени или константи от същия тип, които се наричат *rvalue*. Мястото в паметта, в което е записана *rvalue* се нарича адрес на променливата или още *lvalue*. Ако имаме дефиницията

```
1 int i = 1024;
```

тогава стойността на променливата `i` (нейното *rvalue*) е 1024 и тя именува място от паметта (*lvalue*) с размери 4 байта, като *lvalue* е адреса на първия байт. Намирането на адреса на дефинирана променлива става чрез унарния префиксен дясноасоциативен оператор `&`:

Синтаксис:

```
1 // &<променлива>
```

където `<променлива>` е дефинирана променлива.

Семантика: Намира адреса на `<променлива>`.

Операторът `&` не може да се прилага върху константи и изрази, както и върху променливи от тип масив, тъй като те имат смисъла на константни указатели.

Съществува специален константен указател с име `nullptr`, наречен нулев, който може да бъде свързан с всеки указател, независимо от неговия тип.

Дефиниция на променлива от тип указател:

Синтаксис:

```
1 // T* <променлива> [=<стойност>];
```

Семантика: Променливата от тип указател съдържа адрес в паметта на обект от даден тип T. С други думи, тя не съхранява стойност от тип T директно, а сочи към място, където такава стойност се намира. Указателната променлива може да бъде инициализирана с адреса на съществуваща променлива от съответен тип чрез оператора `&` или по-късно да получи адрес чрез динамично заделяне на памет.

Оператор за дереференциране:

Синтаксис:

```
1 // *<указател>;
```

където `<указател>` е променлива от тип указател.

Семантика: Операторът `*` е унарен, префиксен и дясноасоциативен. Чрез него се достъпва стойността, намираща се на адреса, съхраняван в указателя. Операторът се нарича още дереференциране, тъй като "разопакова" указателя и връща стойността, към която той сочи.

Пример:

```
1 int a = 12;
2 int* p = &a; // p е инициализирано с адреса на a
3 double* q = nullptr; // q е инициализирано с нулевия указател
```

Указателната аритметика се извършва върху указатели към елементи от последователна памет (например масиви). Позволява преместване на указателя напред или назад в зависимост от размера на типа, към който сочи.

Пример:

```
1 int a = 5, b = 10;
2 int* p = &a;
3 p++; // премества p към "следващият" int в паметта
4
5 int arr[] = {1, 2, 3};
```

```

6 int* p = arr;
7 cout << *(p + 1); // извежда 2
8 // p + 1 не означава добавяне на 1 байт, а на 1 елемент от типа,
9 // към който сочи p. Например ако p е int*, и int е 4 байта,
10 // тогава p + 1 сочи 4 байта по-напред.
11 ++p; // премества p към arr[1], след това взема стойността => 2
12 ++*p; // увеличава стойността на arr[1] => става 3
13 (*p)++; // отново увеличава arr[1], сега става 4
14
15 int arr[] = {10, 20, 30};
16 p = arr + 2;
17 cout << *--p << endl; // p -> arr[1], извежда 20
18 cout << --*p << endl; // намалява стойността на arr[1] => 19
19 cout << (*p)-- << endl; // извежда 19, после arr[1] става 18

```

Можем да посочим някои еквивалентни изрази в езика:

```

1 // *p == *(p)
2 // p[0] == *p
3 // p[i] == *(p + i)
4 // *p++ == *(p++)
5 // ++*p == ++(*p)
6 // ++p == *(++p)
7 // *(arr + 2) == arr[2]
8 // 2[arr] == arr[2]

```

2.Масиви - основни операции и алгоритми

Логическо описание

Масивът е крайна редица от фиксиран брой елементи от един и същ тип. Към всеки елемент от редицата е възможен пряк достъп, който се осъществява чрез индекс. Операциите включване и изключване на елемент в/от масива са недопустими (защото е статична структура от данни).

Физическо представяне

Елементите на масива се записват последователно в паметта на компютъра, като за всеки елемент на редицата се отделя определено количество памет.

Тип масив

Типът масив се определя чрез задаване на типа и броя на елементите на редицата, определяща масив. Нека T е име или дефиниция на произволен тип, различен от псевдоним, **void** и функционален. За типа T и константния израз от интегрален или изброен тип с положителна стойност $size$, $T[size]$ е тип масив от $size$ елемента от тип T . Елементите се индексират от 0 до $size - 1$. T се нарича базов тип за типа масив, а $size$ – горна граница.

Примери:

```

1 // Декларация на масив
2 int arr[5]; // масив от 5 елемента от тип int, индексирани от 0
3 // до 4
4 double arr[10]; // масив от 10 елемента от тип double,
5 // индексирани от 0 до 9
6
7 // Инициализация на масив
8 int arr[3] = {1, 2, 3};
9 int arr[] = {4, 5, 6, 7};
10
11 // Промяна на стойности
12 arr[1] = 10;
13
14 // Въвеждане на масив
15 for (int i = 0; i < n; i++) {
16     cin >> arr[i];
17 }
18
19 // Извеждане на масив
20 for (int i = 0; i < n; i++) {
21     cout << arr[i] << " ";
22 }

```

Пример:

```

1 // Динамично заделяне на масив
2 int* arr = new int[5]; // масив от 10 елемента от тип int
3
4 // Освобождаване на заделената памет
5 delete[] arr;

```

Алгоритми за търсене:

```

1 // Линейно търсене, Сложност: O(n)
2 int linearSearch(int arr[], int n, int x) {
3     for(int i = 0; i < n; ++i) {
4         if (arr[i] == x) {
5             return i;
6         }
7     }
8
9     return -1;
10 }
11
12 // Двойчно търсене, Сложност: O(log n)
13 int binarySearch(int arr[], int n, int x) {
14     int left = 0, right = n - 1;

```

```

15
16     while (left <= right) {
17         int mid = (left + right) / 2;
18
19         if (arr[mid] == x) {
20             return mid;
21         }
22
23         arr[mid] < x ? left = mid + 1 : right = mid - 1;
24     }
25
26     return -1;
27 }

```

Алгоритми за сортиране:

```

1  // Пряка селекция, Сложност:  $O(n^2)$ 
2  void selectionSort(int arr[], int n) {
3      for (int i = 0; i < n - 1; i++) {
4
5          int minI = i;
6
7          for (int j = i + 1; j < n; j++) {
8              if (arr[minI] > arr[j]) {
9                  minI = j;
10             }
11         }
12
13         int temp = arr[minI];
14         arr[minI] = arr[i];
15         arr[i] = temp;
16     }
17 }
18
19 // Метод на мехурчето, Сложност:  $O(n^2)$ 
20 void bubbleSort(int arr[], int n) {
21     for (int i = 0; i < n - 1; i++) {
22         for (int j = 1; j < n - 1 - i; j++) {
23             if (arr[j] < arr[j - 1]) {
24                 int temp = arr[j];
25                 arr[j] = arr[j - 1];
26                 arr[j - 1] = temp;
27             }
28         }
29     }
30 }

```

```

31
32 // Вмъкване, Сложност:  $O(n^2)$ 
33 void insertionSort(int arr[], int n) {
34     for (int i = 1; i < n; i++) {
35         int j = i, key = arr[i];
36
37         while (j > 0 && key < arr[j - 1]) {
38             arr[j] = arr[j - 1];
39             --j;
40         }
41
42         arr[j] = key;
43     }
44 }

```

Многомерните масиви представляват масиви от масиви. Най-често се използват двумерни масиви, които могат да се разглеждат като таблици или матрици.

Примери:

```

1 // Декларация и инициализация на матрица
2 int matrix[2][3] = {
3     {1, 2, 3},
4     {4, 5, 6}
5 };
6
7 cout << matrix[0][1]; // Извежда стойността 2
8
9 // Въвеждане на матрица
10 for (int i = 0; i < n; i++) {
11     for (int j = 0; j < m; j++) {
12         cin >> matrix[i][j];
13     }
14 }
15
16 // Обхождане на матрица
17 for (int i = 0; i < n; i++) {
18     for (int j = 0; j < m; j++) {
19         if (i == j) {
20             // обхождане по главен диагонал
21         }
22     }
23 }
24
25 // Извеждане на матрица

```

```

26 for (int i = 0; i < n; i++) {
27     for (int j = 0; j < m; j++) {
28         cout << matrix[i][j] << " ";
29     }
30     cout << endl;
31 }

```

3. Рекурсия

Функция, която се обръща пряко или косвено към себе си, се нарича рекурсивна. Програма, съдържаща рекурсивна функция е рекурсивна. Според начина на извикване рекурсията бива пряка и косвена, а според структурата на извикванията - линейна и разклонена.

Пример:

```

1 int fact(int n) {
2     if (n <= 1) return 1; // базов случай
3     return n * fact(n - 1); // рекурсивен случай
4 }

```

Рекурсивната функция **fact**, приложена към естествено число, връща факториела на това число. Стойността на функцията се определя посредством обръщение към самата функция в оператора **return n * fact(n - 1)**. Нека проследим хода на рекурсивната функция при примерно извикване:

```

1 cout << n << "!=" << fact(n);

```

Този вид рекурсия се нарича **пряка рекурсия**, защото директно извиква сама себе си. Другият вид рекурсия е **косвената рекурсия**, при която дадена функция извиква друга функция, която на свой ред извиква първата функция обратно, създавайки по този начин рекурсивен цикъл.

Пример:

```

1 #include <iostream>
2 using std::cout; using std::endl;
3
4 void fA(int n);
5 void fB(int n);
6
7 void fA(int n) {
8     if (n > 0) {
9         cout << "fA: " << n << endl;
10        fB(n - 1); // Извикваме fB с n-1
11    }
12 }

```



```

13
14 void fB(int n) {
15     if (n > 0) {
16         cout << "fB: " << n << endl;
17         fA(n - 1); // Извикваме fA с n-1
18     }
19 }
20
21 int main() {
22     int n = 5;
23     fA(n); // Стартиране на рекурсията
24     return 0;
25 }

```

Линейна рекурсия е налице, когато всяко рекурсивно извикване води до максимум едно допълнително рекурсивно извикване, създавайки линейна верига от извиквания.

Пример:

```

1 int fact(int n) {
2     if (n == 0) return 1;
3     return n * fact(n - 1);
4 }

```

Разклонена рекурсия имаме, когато всяко рекурсивно извикване води до повече от едно допълнително рекурсивно извикване, създавайки дървовидна структура от извиквания.

Пример:

```

1 int fib(int n) {
2     if (n <= 1) return n;
3     return fib(n - 1) + fib(n - 2);
4 }

```

Обектно-ориентирано програмиране

16. ООП. Основни принципи. Класове и обекти. Наследяване и капсулация.

1. Абстракция със структури от данни. Класове и обекти. Видове конструктори. RAII. Методи

Абстракция със структури от данни е подход, при който методите за използване на данните са разделени от методите за тяхното представяне. Тя

е един от фундаменталните принципи на обектно-ориентираното програмиране и се реализира чрез класове. Програмите се конструират така, че да работят с абстрактни данни (данни с неуточнено представяне). След това представянето се конкретизира с помощта на множество функции, наречени конструктори, селектори и предикати, които реализират "абстрактните данни" по конкретен начин.

Пример:

```
1 #include <iostream>
2 using std::cout; using std::endl;
3
4 class A {
5     public:
6         // Публичен метод, който предоставя интерфейс към
7         // скритата логика. Потребителят на класа използва само
8         // този метод, без да знае какво има вътре.
9         void execute() {
10             logic();
11         }
12
13     private:
14         // Скрита логика, недостъпна извън класа.
15         // Детайлите на реализацията са капсулирани.
16         void logic() {
17             cout << "Internal operations" << endl;
18         }
19 };
20
21 int main() {
22     A a;
23     a.execute(); // Потребителят извиква само публичния метод
24     return 0;
25 }
```

Класовете са основни програмни единици в обектно-ориентираните езици, представляващи съвкупност от променливи и функции, които са обвързани в логическа структура и работят заедно. Те служат като модели (шаблони) за представяне на реални обекти, описвайки атрибути (свойства) и методи (поведение) на обектите.

Декларацията на клас се състои от заглавие и тяло. Заглавието започва с ключовата дума **class**, следвана от идентификатор, задаващ името на класа. Тялото съдържа декларации на компонентите на класа (член-данни и член-функции). Декларациите са заградени във фигурни скоби, следвани от знака ; или списък от обекти следван от ;.

Пример:

```
1 class Date {  
2     // Тяло на класа  
3 };
```

Обектите са екземпляри на даден клас. Връзката между клас и обект е подобна на връзката между тип данни и променлива величина, но за разлика от обикновените променливи, обектите се състоят от множество компоненти (член-данни и член-функции). Ако в клас явно не е дефиниран конструктор, **дефиницията на обект** от този клас не трябва да съдържа инициализация с явно обръщение към конструктор, т.е. трябва да има вида: <име_на_клас><обект>;.

Пример:

```
1 Date d;
```

В този случай, автоматично в класа се създава подразбиращ се конструктор и инициализацията на обекта се осъществява чрез него. Този конструктор изпълнява редица действия, като заделяне на памет за обектите, инициализиране на системни променливи и т.н.

Създаването на обекти е свързано със заделяне на памет, запомняне на текущо състояние, задаване на начални стойности и други дейности, които се наричат инициализация на обектите. Тя се изпълнява от специален вид член-функции на класовете - **конструкторите**. Конструкторът е член-функция, която притежава повечето характеристики на другите член-функции и има редица особености като:

- името на конструктора съвпада с името на класа
- типът на резултата не се указва явно
- изпълнява се автоматично при създаване на обекти
- не може да се извиква явно

В клас може явно да не е дефиниран конструктор, но може да са дефинирани и няколко конструктора. Всички те имат едно и също име - името на класа, но трябва да се различават по броя и/или типа на параметрите си. Такива конструктори се наричат предефинирани. При създаването на обект от клас се изпълнява само един от тях. **Основните видове конструктори** са:

- Конструктор по подразбиране
- Конструктор с параметри
- Конструктор за копиране

Ако в даден клас не е дефиниран конструктор, автоматично се създава т.н. конструктор по подразбиране. Той реализира множество от действия като заделяне на памет за член-данните на обект, инициализиране на системни променливи и др. Този конструктор може да бъде предефиниран, като за целта е необходимо в класа да бъде дефиниран конструктор без параметри.

Пример:

```
1 Date::Date() {  
2     day = 8;  
3     month = 7;  
4     year = 2025;  
5 }
```

Инициализацията на новосъздаден обект на даден клас може да зависи от друг обект на същия клас. За да се укаже такава зависимост, се използват операторът за присвояване или кръгли скоби. Конструкторът за присвояване е специален конструктор, който поддържа формален параметър от тип

```
1 // const <име_на_клас>&
```

Ако в един клас явно не е дефиниран конструктор за присвояване, компилаторът автоматично създава такъв, в момента когато новосъздаден обект се инициализира с обект, намиращ се от дясната страна на оператора за присвояване или кръгли скоби. Този конструктор за присвояване се нарича конструктор за копиране.

Пример:

```
1 Date::Date(const Date& other) {  
2     day = other.day;  
3     month = other.month;  
4     year = other.year;  
5 }
```

RAII (Resource Acquisition Is Initialization) е основен принцип в езика C++, който се използва за надеждно управление на динамични ресурси. Идеята се състои в това, че когато даден ресурс бъде заделен в конструктора на обект, неговото освобождаване трябва да се извършва автоматично, без нужда от ръчна намеса. Това се постига чрез **деструктор** – специален метод в класа, който се извиква автоматично, когато обектът излезе от обхват или бъде унищожен. Благодарение на този механизъм се гарантира, че ресурсът ще бъде освободен правилно и навреме, дори ако програмата напусне текущия блок поради грешка или изключение. RAII позволява обектите да поемат отговорността за ресурсите, които използват, и така се

избягват често срещани проблеми като течове на памет и забравени файлови дескриптори. Когато даден клас използва динамична памет, конструкторът ѝ я заделя с `new`, а деструкторът се грижи тя да бъде освободена с `delete`. Потребителят на класа не е нужно да се занимава с управлението на паметта – това става прозрачно и безопасно чрез конструктора и деструктора. RAII е в основата и на стандартните контейнери и ресурсоуправляващи обекти в C++, като `std::vector` и `std::string`, които автоматично се грижат за ресурсите, които използват.

Пример:

```
1 class RAII {
2     public:
3         // Конструктор: заделя ресурс (динамична памет)
4         RAII() {
5             arr = new int[5]; // заделяме масив с 5 елемента
6             cout << "Memory has been allocated.";
7         }
8
9         // Деструктор: освобождава ресурса
10        ~RAII() {
11            delete[] arr;
12            cout << "Memory has been deallocated.";
13        }
14        private:
15            int* arr; // динамичен масив
16 };
17
18 int main() {
19     RAII obj; // при създаване на обекта се заделя памет
20
21     // тук използваме масива, ако има нужда
22
23     // когато 'obj' излезе от обхват (в края на main),
24     // автоматично се вика деструкторът
25
26     return 0;
27 }
```

2.Наследяване. Производни и вложени класове. Капсулация. Статични полета и методи.

Наследяването е механизъм за създаване на нови класове чрез използване на компоненти и поведение на вече съществуващи класове. Класовете, които са вече дефинирани се наричат базови, а новосъздадените - **производни**. Производният клас може да наследи компонентите на един или

няколко базови класа. В първия случай наследяването се нарича единично, а във втория - множествено. Ако множество от класове имат общи член-данни и методи, тези общи части могат да се обособят като основни класове, а всяка от останалите части да се дефинира като производен клас на съответните основни класове. По този начин се спестява памет, тъй като се избягва многократното описание на едни и същи програмни фрагменти. При конструирането на производни класове е достатъчно да се разполага само с обектните модули на основните класове, а не с техния програмен код. Това позволява да бъдат създавани библиотеки от класове, които да бъдат използвани при създаването на производни класове. Тези предимства, а също и възможността за реализиране на полиморфизъм, мотивират въвеждането на наследяване и на производни класове. Подобно на обикновените, производните класове се дефинират, като се декларира класът и се дефинират неговите методи.

Пример:

```
1 #include <iostream>
2 #include <cstring>
3 using std::cout; using std::endl;
4
5 // Базов клас
6 class Person {
7     public:
8         Person(const char* n) {
9             name = new char[strlen(n) + 1];
10            strcpy(name, n);
11        }
12
13        void print() const {
14            cout << "Name: " << name << endl;
15        }
16
17        ~Person() { delete[] name; }
18    private:
19        char* name;
20 };
21
22 // Производен клас
23 class Student : public Person {
24     public:
25         Student(const char* n, long id) : Person(n) {
26             studentID = id;
27         }
28
29         void print() const {
```

```

30         Person::print(); // извикваме печатане на име
31         // от базовия клас
32         cout << "ID: " << studentID << endl;
33     }
34     private:
35         long studentID;
36 };
37
38 int main() {
39     Student s("Ivan", 81992);
40     s.print();
41
42     return 0;
43 }

```

Вложен клас е клас, дефиниран вътре в друг клас. Той служи за логическа организация на кода, когато даден клас е тясно свързан с друг и не се използва самостоятелно. В C++ вложените класове имат достъп само до публичните и статичните членове на обграждащия клас, освен ако не са специално декларирани за достъп.

Пример:

```

1  class Outer {
2      public:
3          int a = 10;
4          static int b;
5
6          void f() {
7              // Външният клас може да достъпва данните на
8              // вътрешния клас само ако те са публични
9              i.show();
10         }
11
12         // Клас, който се вижда само от Outer
13         class Inner {
14             public:
15                 int x = 5;
16                 void show() {
17                     // Вътрешният клас може да достъпва само
18                     // статични данни на външния клас
19                     cout << a; // Не е позволено
20                     cout << b; // Позволено
21                 }
22         };
23

```

```

24         Inner i;
25     };
26
27     int Outer::b = 20;
28
29     int main() {
30         Outer o;
31         o.f();
32
33         return 0;
34     }

```

В C++ достъпът до членовете на даден клас се управлява чрез спецификатори за достъп (**public**, **protected**, **private**). Те определят кои части от програмата имат право да достъпват съответните полета и методи:

- *public* - достъпни от всяка точка на програмата
- *protected* - достъпни само в класа и неговите производни класове
- *private* - достъпни само в рамките на същия клас

При наследяване възниква въпросът какво става с достъпа до членовете на базовия клас в производния клас. Това зависи както от спецификаторите на достъп на членовете в базовия клас, така и от вида на наследяване, използван при дефиницията на производния клас. Следната таблица обобщава как достъпът до членовете на базовия клас се променя в зависимост от типа на наследяване, т.е. как се осъществява **достъпът до наследните компоненти**:

Вид	public	protected	private
public	остава public	става protected	става private
protected	остава protected	остава protected	става private
private	недостъпен	недостъпен	недостъпен

Пример:

```

1  class Base {
2      private:
3          int a;
4      protected:
5          int b;
6      public:
7          int c;
8
9      void setBase() {
10         a = 10;
11         b = 20;
12         c = 30;

```



```

13     }
14 };
15
16 class Derived : Base {
17     public:
18         void setDerived() {
19             a = 1; // Не може да се достъпи
20             b = 2; // Може да се достъпи
21             c = 3; // Може да се достъпи
22         }
23 };
24
25 int main() {
26
27     Base x;
28     x.a = 15; // Не може да се достъпи
29     x.b = 30; // Не може да се достъпи
30     x.c = 90; // Може да се достъпи
31
32     return 0;
33 }

```

Капсулацията е един от основните принципи в обектно-ориентираното програмиране. Тя представлява механизъм за ограничаване на достъпа до вътрешните компоненти на обектите и се използва, за да се контролира как и дали външният свят може да взаимодейства с тях. Чрез капсулацията се постига ясно разграничение между това, което един обект предоставя като интерфейс, и това, което крие като вътрешна логика и реализация.

Капсулацията се реализира чрез използването на спецификатори за достъп – **public**, **protected** и **private**. Обичайна практика е член-данните (променливите) на класа да бъдат декларирани като **private**, за да не могат да бъдат достъпвани директно отвън. Вместо това се предоставят специални методи, наречени селектори (**get** методи) и мутатори (**set** методи), чрез които се осъществява контролирано четене и писане на стойности. По този начин вътрешното състояние на обекта е защитено от неволни или злонамерени промени, което увеличава надеждността и предсказуемостта на програмата.

Скриването на информацията, което често се разглежда заедно с капсулацията, означава точно това – детайлите по вътрешната реализация на един клас да останат невидими за кода, който го използва. От гледна точка на потребителя на даден клас, е важно да се знае само какви операции могат да се извършват с обекта, а не как точно те се реализират. Това позволява по-лесна поддръжка и развитие на кода, тъй като вътрешната реализация може да се променя, без това да засяга останалите части на програмата.

Пример:

```
1 class BankAccount {
2     public:
3         // Публичен метод за контролирано внасяне на средства
4         void deposit(double amount) {
5             if (amount > 0) balance += amount;
6         }
7
8         // Публичен метод, чрез който се предоставя само четене
9         // на баланс по сметка
10        double getBalance() const {
11            return balance;
12        }
13    private:
14        double balance; // скрито поле - не може да бъде
15        // достъпвано директно отвън
16};
```

Всеки обект на клас има свое копие на член-данните на класа, но има случаи при които е необходимо всички обекти на клас да споделят една или повече негови член-данни. За да се създадат споделени от всички обекти на клас член-данни, последните се декларират като **статични**. Това се осъществява чрез ключовата дума **static**. Статичните член-данни имат следните характеристики:

- памет за този вид член-данни се заделя еднократно и всички обекти на класа имат достъп до нея. Промяната ѝ от един обект е валидна за всеки друг обект
- памет за статичните член-данни се заделя не в стекова рамка, а в областта за статичните данни. Това им определя статут на външни променливи, поради което статичните член-данни могат да се използват и самостоятелно, а не само свързано с обектите на класа
- достъп до статичните член-данни на клас може да се осъществява от всяка външна функция чрез използване на пълното им име, стига статичните член-данни да са дефинирани в **public** секция на класа. При този достъп не е нужно статичните член-данни да са свързани с обект на класа
- статичните член-данни се задават в декларацията на класа. Преди използването им трябва задължително да бъдат дефинирани с инициализация. Дефиницията им се осъществява извън декларацията на класа

Пример:

```
1 class StaticData {
```

```

2 public:
3     StaticData(int m = 0) {
4         x = m;
5         n++;
6     }
7     ~StaticData() {
8         n--;
9     }
10    void print() const {
11        cout << x << " " << &x << " " << n << " "
12        << &n << endl;
13    }
14 private:
15     int x;
16     static int n; // статична член-данна
17 };
18
19 int StaticData::n = 0;
20
21 int main() {
22
23     StaticData s1(10); s.print();
24     StaticData s2(20); s.print();
25     StaticData s3(30); s.print();
26     StaticData s4(40); s.print();
27
28     return 0;
29 }

```

За статичната член-данна `n` не се отделя памет при създаването на обектите `s1`, `s2`, `s3`, `s4` след изпълнение на еднопараметричния конструктор. Памет за тази член-данна е отделена в областта за статични данни. Тази памет е обща за четирите обекта и последователно получава стойностите 0, 1, 2, 3, 4. Статичните член-данни имат за област - тази на класа, в който са дефинирани. Това е причината достъпът до тях да е чрез оператора `::`. Освен статични член-данни в класовете могат да бъдат дефинирани и статични член-функции. Това се осъществява отново чрез ключовата дума `static`, която се поставя пред типа в прототипа на член-функцията. Особеност на статичните член-функции е, че те нямат неявен параметър, т.е. за тях не се дефинира указателят `this`. Това е причината, поради която:

- статичните член-функции на клас нямат пряк достъп до нестатичните компоненти на обектите, чрез които са били извиквани, т.е. в тялото на статичен метод могат да се използват само статични компоненти. Нестатични компоненти могат да се извикват с помощта на указател, подобен на указателя `this`.

- статичните член-функции на класовете могат да бъдат извиквани, както чрез обекти на класа, така и самостоятелно. Самостоятелното извикване изисква да се използват пълните имена на тези методи.

Пример:

```

1 #include <iostream>
2 using std::cout; using std::endl;
3
4 class StaticMethod {
5     public:
6         StaticMethod(int = 0);
7         static int func(int);
8     private:
9         int x;
10 };
11
12 StaticMethod::StaticMethod(int m) {
13     x = m;
14 }
15
16 int main() {
17
18     StaticMethod s;
19     cout << StaticMethod::func(5) << endl;
20
21     return 0;
22 }

```

Методите, декларирани като статични, имат следните ограничения:

- не могат да бъдат виртуални
- не могат да бъдат член-функции за достъп
- модификаторът `static` не е основание за различаване на две член-функции с еднакви имена

17. ООП. Подтипов и параметричен полиморфизъм. Множествено наследяване.

1. Виртуални функции и подтипов полиморфизъм. Абстрактни методи и класове. Масиви от обекти и от указатели към обекти.

Полиморфизмът е важна характеристика на обектно-ориентираното програмиране, която се изразява в това, че едни и същи действия се реализират по различен начин в зависимост от обектите, над които се прилагат, т.е. действията са полиморфни (с много форми). Полиморфизмът е свойство

на член-функциите на обектите и се реализира чрез виртуални функции. За да се реализира полиморфно действие, класовете над които то ще се прилага, трябва да имат общ родител, т.е. да бъдат производни на един и същ клас. В този клас трябва да бъде дефиниран виртуален метод, съответстващ на полиморфното действие. Във всеки от производните класове този метод може да бъде предефиниран съобразно особеностите на класа. Активирането на полиморфното действие се осъществява чрез указател към базовия клас или чрез псевдоним на обект на базовия клас. В зависимост от типа на обекта, към който сочи указателят, ще бъде изпълняван един или друг виртуален метод. Ако класовете, над които ще се реализира полиморфното действие, нямат общ родител, такъв може да бъде създаден изкуствено чрез дефиниране на т.н. абстрактен клас.

В езика за програмиране C++, при извикване на функции с едно и също име, е имплементиран механизъм за разпознаване коя функция да се изпълни. Той се изразява в следното: по време на компилация се сравняват формалните с фактическите параметри в обръщението и по правилото за най-добро съвпадане се избира необходимата функция. След заместване на формалните с фактическите параметри, се изпълнява тялото на функцията. При член-функциите на йерархия от класове конфликтът между имената на наследените и собствените методи с един и същ тип на резултата и с един и същи типове на явно указаните формални параметри се разрешава също по време на компилация чрез правилото на локалния приоритет и чрез явно посочване на класа, към който принадлежи методът. В тези два случая, тъй като разрешаването на конфликтите приключва по време на компилация и не може да бъде променяно по време на изпълнение на програмата, се казва че се осъществява статично свързване. Езикът поддържа още един механизъм за разрешаване на връзките, наречен **динамично свързване**. При него изборът на функция, която трябва да се изпълни, се осъществява по време на изпълнение на програмата. Динамичното свързване капсулира детайлите в реализацията на йерархията. При него не се налага преобразуване на типове, текстовете на програмите се опростяват, а промени се налагат много по-рядко. Прилагането на механизма на динамичното свързване се осъществява над специални член-функции на класове, наречени **виртуални функции** (методи). Те се декларираат чрез поставяне на ключовата дума **virtual** пред декларацията им:

```
1 virtual void print();
```

Чисто виртуални функции наричаме функции, които не са дефинирани, а само декларирани. Синтаксисът за тях е следния:

```
1 virtual void print() = 0;
```

Пример:

```
1 #include <iostream>
```

```

2 using std::cout; using std::endl;
3
4 class Device {
5     public:
6         void info() const {
7             cout << "Device: None" << endl;
8         }
9 };
10
11 class Laptop : public Device {
12     public:
13         void info() const {
14             cout << "Device: Laptop" << endl;
15         }
16 };
17
18 int main() {
19     Device* d = new Laptop();
20     d->info(); // тук ще се извика Device::info(), а не
21               // както очакваме Laptop::info()
22
23     return 0;
24 }

```

Член-функцията `info()` на всеки от класовете извежда общата информация `Device` и специфичната за всеки клас информация - определяща вида на устройството. Поведението което целим да постигнем се осъществява с ключовата дума `virtual` пред метода на базовия клас. Тогава ще се извика очаквания метод:

```

1 ...
2 ...
3
4 class Device {
5     public:
6         virtual void info() const {
7             cout << "Device: None" << endl;
8         }
9 };
10
11 ...
12
13 int main() {
14     Device* d = new Laptop();
15     d->info(); // така извикваме Laptop::info()
16 }

```

```
17     return 0;
18 }
```

Подтиповият полиморфизъм позволява прилагането на едни и същи операции върху обекти от различни класове, които наследяват общ базов клас. Основен механизъм за реализиране на подтиповия полиморфизъм в C++ е използването на виртуални функции. Използвайки указатели или референции към базовия клас, можем да извикваме методи, чиято конкретна реализация зависи от типа на обекта, към който всъщност сочи указателят — това се нарича динамично свързване.

Пример:

```
1  #include <iostream>
2  using std::cout; using std::endl;
3
4  class Device {
5  public:
6      virtual void start() const {
7          cout << "Device starting..." << endl;
8      }
9
10     virtual ~Device() {} // Виртуален деструктор за
11                          // правилно освобождаване на памет
12 };
13
14 class Phone : public Device {
15 public:
16     void start() const override {
17         cout << "Phone is booting up..." << endl;
18     }
19 };
20
21 class Laptop : public Device {
22 public:
23     void start() const override {
24         cout << "Laptop is powering on..." << endl;
25     }
26 };
27
28 int main() {
29     Device* devices[3];
30     devices[0] = new Device();
31     devices[1] = new Phone();
32     devices[2] = new Laptop();
33 }
```

```

34     for (int i = 0; i < 3; ++i) {
35         devices[i]->start(); // динамично свързване
36         delete devices[i];
37     }
38
39     return 0;
40 }

```

В горния пример методът `start()` се извиква чрез указател към базовия клас `Device`, но реално се изпълнява версията, съответстваща на конкретния обект (`Phone`, `Laptop`), към който сочи указателят. Това е пример за подтипов полиморфизъм, реализиран чрез виртуални функции.

Клас, в който е декларирана поне една чисто виртуална функция, се нарича **абстрактен**. Абстрактните класове имат следните свойства:

- не могат да се създават обекти от абстрактен клас, но могат да се дефинират указатели и псевдоними от такъв клас
- чисто виртуалните член-функции задължително трябва да бъдат предефинирани в класовете, производни на абстрактния клас или да бъдат обявени за чисто виртуални в тях

Абстрактните класове са предназначени да са базови на други класове. Полиморфизмът, с помощта на абстрактните класове, позволява създаването на хетерогенни структури от данни, т.е. елементите на които са от различен тип (това могат да бъдат стек, опашка, дърво, граф и др.).

Пример:

```

1  class Device {
2      public:
3          // Чисто виртуална функция
4          virtual void start() const = 0;
5
6          // Чисто виртуална функция
7          virtual void shutdown() const = 0;
8
9          // Виртуален деструктор
10         virtual ~Device() {}
11 };

```

2.Параметричен полиморфизъм. Шаблони на функция и клас.

Параметричният полиморфизъм позволява писането на код, който работи с различни типове данни, без да се променя неговата структура. В C++ той се реализира чрез шаблони (templates) на функции и класове.

Пример за шаблон на функция:


```

1 #include <iostream>
2 using std::cout; using std::endl;
3
4 template <typename T>
5 T maximum(T a, T b) {
6     return (a > b) ? a : b;
7 }
8
9 int main() {
10     cout << maximum(5, 10) << endl;           // int
11     cout << maximum(3.14, 2.71) << endl;       // double
12     cout << maximum('a', 'z') << endl;        // char
13
14     return 0;
15 }

```

Функцията `maximum` приема два параметъра от един и същи тип и връща по-големия от тях. Типът се определя по време на компилация на база аргументите, подадени при извикване.

Шаблон на клас:

```

1 #include <iostream>
2 #include <string>
3 using std::cout; using std::endl; using std::string;
4
5 template <typename T>
6 class Box {
7     public:
8         Box(T v) : value(v) {}
9         void print() const {
10             cout << "Value: " << value << endl;
11         }
12     private:
13         T value;
14 };
15
16 int main() {
17     Box<int> intBox(42);
18     Box<string> strBox("Hello");
19
20     intBox.print();
21     strBox.print();
22
23     return 0;
24 }

```

Класът `Box` е шаблонен клас, който може да съхранява стойности от произволен тип. Това позволява създаването на обекти от един и същи клас, но с различни типове.

Параметричният полиморфизъм улеснява повторното използване на код и повишава неговата гъвкавост.

3. Множествено наследяване.

Наследяване наричаме множествено, когато производният клас наследява директно повече от един базов клас. Чрез това наследяване могат да се изграждат програми с мрежова структура.

Пример:

```
1 #include <iostream>
2 using std::cout; using std::endl;
3
4 class Device {
5     public:
6         virtual ~Device() = default;
7
8         // Чисто виртуална функция за стартиране на устройството
9         virtual void start() const = 0;
10
11        // Чисто виртуална функция за типа на устройството
12        virtual const char* getType() const = 0;
13 };
14
15 class Phone : virtual public Device {
16     public:
17         void start() const override {
18             cout << "Starting the phone!" << endl;
19         }
20
21         const char* getType() const override {
22             return "Phone";
23         }
24 };
25
26 class Laptop : virtual public Device {
27     public:
28         void start() const override {
29             cout << "Starting the laptop!" << endl;
30         }
31
32         const char* getType() const override {
```

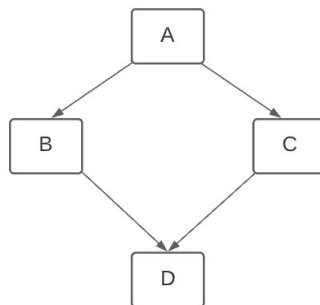
```

33         return "Laptop";
34     }
35 };
36
37 // Множествено наследяване
38 class Smartphone : public Phone, public Laptop {
39     public:
40         void start() const override {
41             Phone::start();
42             Laptop::start();
43         }
44
45         const char* getType() const override {
46             return "Smartphone";
47         }
48 };
49
50 int main() {
51
52     Smartphone s;
53
54     cout << "Device type: " << s.getType() << endl;
55     s.start();
56
57     return 0;
58 }

```

Диамантен проблем

Диамантения проблем възниква при множествено наследяване, когато два класа наследяват от един общ базов клас, а трети клас наследява от тях двамата. Получава се "диамантена" структура:



По-долу демонстрираме проблема, реализиран на C++:

```

1 #include <iostream>

```

```

2 using std::cout; using std::endl;
3
4 class A {
5     public:
6         void hello() {
7             cout << "Hello from A" << endl;
8         }
9 };
10
11 class B : public A {};
12 class C : public A {};
13 class D : public B, public C {};
14
15 int main() {
16     D obj;
17     // obj.hello(); // Двусмислено - компилаторът ще даде грешка!
18     obj.B::hello(); // Можем да уточним по кой път
19 }

```

Решението - виртуално наследяване:

```

1 ...
2 class B : virtual public A {};
3 class C : virtual public A {};
4 class D : public B, public C {};
5 ...
6     obj.hello(); // Вече не е двусмислено - има само едно
7     // копие на A

```

Когато използваме **virtual**, C++ създава само едно общо копие на A. Това копие се споделя от B и C – така D има едно, ясно определено наследяване.

Структури от данни и алгоритми

18. Структури от данни. Стек, опашка, списък, дърво. Основни операции върху тях. Реализация.

1. Структури от данни

Структура от данни наричаме организирана информация, която може да бъде описана, създадена и обработена с помощта на програма. За да се определи една структура от данни, е необходимо да се опише:

- логическо описание на структурата - описание на базата на декомпозицията ѝ на по-прости структури и декомпозицията на операциите над структурата с по-прости операции

- физическо представяне на структурата - описание на методи за представяне на структурата в паметта на компютъра

Структурите от данни се делят на:

- линейни - елементите са подредени последователно (списък, стек, опашка)
- нелинейни - елементите са подредени йерархично или мрежово (дърво, граф)

Нека T е даден тип данни.

2.Списък

Логическо описание

Списък с елементи от тип T е линейна динамична структура от данни. Определя се като крайна редица от елементи от тип T . Елементът, който стои в началото на линейният списък, се нарича първи, глава или начален елемент. Останалите елементи на редицата също образуват списък, който се нарича подсписък или опашка на списъка. Над списък могат да се изпълняват следните операции:

- създаване на празен списък
- определяне дали списък е празен
- добавяне на елемент към списък
- премахване на елемент от списък, ако не е празен
- достъп до елемент на списък, ако не е празен

Операциите добавяне, отстраняване и достъп се извършват в произволно място на списъка. Достъпът е пряк само до първия елемент и е последователен до елементите на опашката на списъка. С цел улесняване на изпълнението на операциите повечето реализации допускат пряк достъп до последния елемент на списъка, както и до произволен елемент, ако е известен адресът му.

Списък с една и две връзки. Характеристики.

Използват се главно 2 начина за физическо представяне на линеен списък в паметта на компютъра - последователно и свързано. Последователното представяне не се използва заради добре развитите средства за динамично разпределение на паметта. Свързаното представяне се реализира върху списък с една или с две връзки. Основната разлика между тях е в броя на връзките (указателите), които се съдържат във всеки елемент на списъка.

Списък с една връзка, наричан още едносвързан списък, е структура, при която всеки възел съдържа данни и указател към следващия елемент в редицата. В този тип списък връзките са еднопосочни – т.е. можем да обхождаме списъка само в една посока – от началото към края. Началото на списъка се отбелязва с указател към първия възел (т.н. "глава"), а края се разпознава по това, че последният елемент сочи към `nullptr`. Основните операции като добавяне на елемент в началото са бързи и имат сложност $O(1)$, докато добавянето или премахването на елемент на произволна позиция изисква обхождане на списъка и съответно сложност $O(n)$.

Списъкът с две връзки, известен още като двусвързан списък, разширява функционалността на едносвързания, като всеки възел съдържа два указателя – един към следващия и един към предишния елемент. Това позволява списъкът да се обхожда в двете посоки, което значително улеснява операции като премахване на елементи, особено когато елементът е даден чрез указател. В сравнение с едносвързания списък, при двусвързания добавянето както в началото, така и в края на списъка може да се извършва с константна сложност $O(1)$, ако се поддържа указател към края на списъка. Това го прави по-гъвкав, но и по-паметоемък, тъй като за всеки елемент трябва да се съхраняват два указателя. Изборът между едносвързан и двусвързан списък зависи от конкретните нужди на приложението – дали се изисква честа навигация в двете посоки и ефективно премахване, или се търси по-икономична структура, при която се работи основно от едната страна на списъка.

Пример:

```
1 // Възел
2 template <typename T>
3 struct Node {
4     T data;
5     Node<T>* next;
6 };
7 // Член-данни
8 ...
9 private:
10     Node<T> *head, *tail;
```

Сложност

- добавяне на елемент
 - в началото - $O(1)$
 - на някоя позиция - $O(n)$
 - в края - $O(n)$, за двойно свързан - $O(1)$
- премахване на елемент

- в началото - $O(1)$
- на някоя позиция - $O(n)$
- в края - $O(n)$, за двойно свързан - $O(1)$
- намиране на елемент - $O(n)$

3.Стек

Логическо описание

Стек с елементи от тип **T** е линейна динамична структура от данни. Определя се като крайна редица от елементи от тип **T**. Единият край на редицата, в случай че тя не е празна, се нарича връх или начало на стека. Стекът се определя като структура "последен влязъл - пръв излязъл" (*LastInFirstOut – LIFO*). Над стек могат да се изпълняват следните операции:

- създаване на празен стек
- определяне дали стек е празен
- добавяне на елемент във върха на стек
- премахване на елемент от върха на стек, ако не е празен
- достъп до елемента от върха на стек, ако не е празен

Статична, динамична и свързана реализация. Характеристики.

Статична реализация - запазва се блок от паметта, в който стекът да се променя: нараства при добавяне на елементи или намалява при премахване на елементи. При добавяне на елементи, те се поместват в последователни адреси в неизползваната част веднага след върха на стека. Тази операция е възможна до пълното изчерпване на неизползваната част. При отстраняване на елементи се променя указателят към върха на стека. Блокът от паметта, в който се съхранява стекът, се реализира чрез едномерен масив, а указателят към върха на стека - чрез индекс.

Предимства:

- опростена реализация и бърз достъп до елементите чрез индекси - ($O(1)$)

Недостатъци:

- фиксиран капацитет и потенциално неоптимално използване на паметта (резервация на повече от нужното)

Динамична реализация - стекът се представя чрез масив, чийто размер не е фиксиран предварително, а се променя по време на изпълнение. При добавяне на елемент, ако масивът достигне максималния си капацитет, се

създава нов, по-голям масив (обикновено с удвоен размер), в който се копират всички текущи елементи. След това новият елемент се добавя към него. Промяната на размера се осъществява чрез преалокация на паметта, като се запазва линейният достъп чрез индекс. Този вид реализация е характерна за структури като `std::vector` в C++.

Предимства:

- гъвкав размер и запазва структурата на масив (достъп чрез индекс)

Недостатъци:

- при достигане на капацитета се извършва операция по преоразмеряване с времева сложност $O(n)$

Свързана реализация - елементите на стека не се разполагат в последователни адреси в паметта, а са разпръснати на свободни места в паметта. Реализира се като всеки елемент на стека се представя чрез блок с две поля: информационно (съдържа стойността на елемента) и адресно (съдържа адреса на предишния елемент на стека). Адресното поле на последния елемент е `nullptr`. Добавянето на елемент се осъществява чрез създаване на нов блок. Премахването на елемент се реализира като в помощна променлива се записва информационното поле на блока, представлящ върха на стека, указателят `top` се свързва с адреса от адресното поле, след което се разрушава блока от върха на стека.

Предимства:

- динамичен размер и лесно добавяне/премахване на елементи - $O(1)$

Недостатъци:

- допълнителна памет за указатели, по-бавен достъп до произволни елементи, по-сложна реализация и управление на памет

Пример:

```
1 // Възел
2 template <typename T>
3 struct Node {
4     T data;
5     Node<T>* next;
6 };
7 // Член-данни
8 ...
9 private:
10     Node<T> *top;
```

Сложност

- добавяне на елемент - $O(1)$

- премахване на елемент - $O(1)$

4.Опашка

Логическо описание

Опашка с елементи от тип **T** е линейна динамична структура от данни. Определя се като крайна редица от елементи от тип **T**. Единият край на редицата, в случай че тя не е празна, се нарича глава или начало на опашката, а другият край се нарича край или задна част на опашката. Опашката се определя като структура "първи влязъл - първи излязъл" (*FirstInFirstOut – FIFO*). Над опашка могат да се изпълняват следните операции:

- създаване на празна опашка
- определяне дали опашка е празна
- добавяне на елемент само в края на опашката
- премахване на елемент само в началото на опашката, ако не е празна
- достъп до елемента в началото на опашката, ако не е празна

Статична, динамична и свързана реализация. Характеристики.

Статична реализация - запазва се блок от паметта, в който опашката да се променя: нараства при добавяне на елементи или намалява при премахване на елементи. Включването на елемент в опашката се осъществява чрез поместването му в последователни адреси в неизползваната част веднага след края на опашката. Съществуват разновидности на това представяне. Една от тях, която най-често се използва е цикличното представяне. При него, при изчерпване на масива, ако има свободна памет в началото му, включването се извършва там. При отстраняване на елементи се променя указателят към началото на опашката и се освобождава паметта, заета от отстранения елемент. Блокът от паметта, в който се съхранява опашката, се реализира чрез едномерен масив, а указателите към началото и след края на опашката - чрез индекси.

Предимства:

- опростена реализация и бърз достъп до елементите чрез индекси - ($O(1)$)

Недостатъци:

- фиксиран капацитет и потенциално неоптимално използване на паметта (резервация на повече от нужното)

Динамична реализация - опашката се представя чрез масив с възможност за динамично преоразмеряване, когато капацитетът бъде достигнат. При добавяне на нов елемент, ако масивът е пълен, се заделя нов по-голям блок

от памет (обикновено с удвоен размер), в който се копират всички текущи елементи от опашката, като след това новият елемент се добавя. По този начин се осигурява по-гъвкаво управление на паметта в сравнение със статичната реализация. Управлението на началото и края на опашката се осъществява чрез два индекса – `front` и `back`, както и при цикличното представяне. Този подход се използва в динамични структури като `std::vector` в C++, при реализиране на опашка чрез масив.

Предимства:

- гъвкав размер и запазва структурата на масив (достъп чрез индекс)

Недостатъци:

- при достигане на капацитета се извършва операция по преоразмеряване с времева сложност $O(n)$

Свързана реализация - аналогична на свързаното представяне на стек. Елементите на опашката не се разполагат в последователни адреси в паметта, а са разпръснати на свободни места в паметта. Всеки елемент на опашката се представя чрез блок с две полета: информационно (съдържащо стойността на елемента) и адресно (съдържащо адреса на следващия елемент от опашката). Адресното поле на последния елемент е означено чрез константата `nullptr`, указателят към началото на опашката е означен с `front`, а указателят към края ѝ - с `back`. Предимства:

- динамичен размер и лесно добавяне/премахване на елементи - $O(1)$

Недостатъци:

- допълнителна памет за указатели, по-бавен достъп до произволни елементи, по-сложна реализация и управление на памет

Пример:

```
1 // Възел
2 template <typename T>
3 struct Node {
4     T data;
5     Node<T>* next;
6 };
7 // Член-данни
8 ...
9 private:
10     Node<T> *front, *back;
```

Сложност

- добавяне на елемент - $O(1)$
- премахване на елемент - $O(1)$

5. Кореново дърво

Логическо описание

Дърво от тип T е нелинейна структура от данни, която е или празна, или е образувана от:

- данна от тип T , наречена корен на двоичното дърво от тип T
- крайно, възможно празно множество от променлив брой несвързани помежду си елементи $T_1 \dots T_n$, които са дървета от тип T , наречени поддървета на дървото от тип T

Линейният списък се нарича изродено дърво, защото може да се разглежда като дърво, всеки възел на което има най-много едно поддърво. Над дърво от тип T , могат да се изпълняват следните операции:

- достъп до връх
- проверка дали дърво е празно
- включване и изключване на поддърво
- включване и изключване на връх от тип T
- обхождане на дърво от тип T

Начини за представяне в паметта

Използват се главно два начина за физическо представяне на кореново дърво от тип T – свързано и верижно. Свързаното представяне се реализира чрез указател към кутия (структура), съдържаща две полета – информационно (стойността на върха) и адресно – указател към линейна структура (списък), съдържаща децата на текущия връх. Всеки от тези елементи в списъка също е корен на поддърво и по същия начин съдържа свои деца.

Пример:

```
1 // Възел
2 template <typename T>
3 struct Node {
4     T data;
5     vector<Node<T>*> children;
6 };
7 // Член-данни
8 ...
9 private:
10     Node<T> *root;
```

Верижното (последователно) представяне се реализира чрез два масива – единият съдържа стойностите на върховете, а другият – структура от спи-

същи или масиви с адреси към децата на съответния връх. i -ят елемент на масива "върхове" съдържа стойността на i -ия връх в дървото. i -ят елемент на масива "деца" съдържа указания към всички преки наследници (деца) на върха с индекс i . Поддържа се и указател (или индекс), който сочи към корена на дървото – това е началната точка на достъп до структурата.

Пример:

```
1 vector<T> nodes; // стойности на възлите
2 vector<vector<int>> children; // списъци с индекси на децата
3 int rootIndex; // индекс на кореновия възел
```

6. Двоично дърво

Логическо описание

Двоично дърво от тип T е нелинейна структура от данни, която е или празна, или е образувана от:

- данна от тип T , наречена корен на дървото от тип T
- двоично дърво от тип T , наречено ляво поддърво на двоичното дърво от тип T
- двоично дърво от тип T , наречено дясно поддърво на двоичното дърво от тип T

Над двоично дърво могат да се изпълняват следните операции:

- достъп до връх
- проверка дали двоично дърво е празно
- включване и изключване на връх
- обхождане на двоично дърво (осъществява се с три действия - обработка на корена, обхождане на ляво поддърво и обхождане на дясно поддърво)

Съществуват шест начина за обхождане на двоично дърво: ляво-корен-дясно, корен-ляво-дясно, ляво-дясно-корен, дясно-корен-ляво, корен-дясно-ляво и дясно-ляво-корен.

Начини за представяне в паметта.

Използват се главно 2 начина за физическо представяне на двоично дърво от тип T - свързано и верижно. Свързаното представяне се реализира чрез указател към кутия с три полета - информационно (стойността на корена) и две адресни (представянията на ляво поддърво и дясно поддърво).

Пример:

```

1 // Възел
2 template <typename T>
3 struct Node {
4     T data;
5     Node<T> *left, *right;
6 };
7 // Член-данни
8 ...
9 private:
10     Node<T> *root;

```

Верижното (последователно) представяне се реализира чрез три масива - за представяне на върховете на дървото, за адресите на лявото и адресите на дясното поддърво. Ролята на адреси се изпълнява от индекси. i -ят елемент на масива "върхове съдържа стойността на връх на двоичното дърво, i -ят елемент на масива "ляво поддърво" съдържа адреса на лявото поддърво на поддървото с корен i -я елемент, а i -ят елемент на масива "дясно поддърво" съдържа адреса на дясното му поддърво. Поддържа се указател, който съдържа адреса на корена.

Пример:

```

1 vector<T> nodes; // стойности на възлите
2 vector<int> left; // индекси на левите деца
3 vector<int> right; // индекси на десните деца
4 int rootIndex; // индекс на кореновия възел

```

Сложност

- добавяне на елемент - $O(\log n)$ за балансирани дървета, $O(n)$ за небалансирани
- премахване на елемент - $O(\log n)$ за балансирани дървета, $O(n)$ за небалансирани
- намиране на елемент - $O(\log n)$ за балансирани дървета, $O(n)$ за небалансирани